Spotter Freight Rate Prediction Assessment Report

Candidate: Kashif Akram

Role: Machine Learning Engineer Assessment

Prediction Target: `posted_rate`

This report presents my approach to the Spotter Freight Rate Prediction assessment, including data preparation, model selection, chronological validation, development benchmark results, error analysis, and final prediction outputs. The report is based on the supplied assessment data, existing solution code, and verified project outputs.

1. Objective and Target

The objective is to predict freight load **posted_rate** for the supplied validation loads. The labeled development data is **train-test.csv**, and the final required prediction file is **validation_predictions.csv** with one predicted rate for each validation **load_id**.

2. Data and Data Preparation

File	Rows	Verified role
train-test.csv	48,000	Labeled development data; includes target posted_rate
validation.csv	12,000	Final validation loads requiring predictions; no posted_rate column
validation-predictions-template.csv	12,000	Template with load_id,predicted_rate
validation_predictions.csv	12,000	Completed final predictions in required format
december-chart-inputs.csv	31	Fixed December rows with predicted_rate filled

- Development missing values verified from CSV: weight=300, market_index=374, posted_rate=0.
- Validation missing values verified from CSV: weight=165, market_index=249.
- Nonpositive recorded weight values verified from CSV: development=292, validation=145.
- In baseline.py, prepare_features uses only the configured model features and converts nonpositive weight values to missing under the default nonpositive-as-missing policy. Numeric missing values are then median-imputed inside the fitted preprocessing pipeline.
- load_id is used for row matching/output construction, not as a predictive feature. market_index and quote_signal are excluded from the selected baseline.

3. Chronological Validation Split

baseline.py implements a chronological development split with **CUTOFF = 2025-10-01**. Rows before the cutoff are fit rows, and rows on or after the cutoff form the October development holdout.

Split	Condition	Rows
Fit	date before 2025-10-01	43,147
Holdout	date on/after 2025-10-01	4,853

predict.py then fits the selected baseline on the full labeled development data and predicts every row in validation.csv. validation.csv is not used for model selection because it does not contain posted_rate.

4. Selected Baseline Model

The selected model is a scikit-learn Pipeline with a ColumnTransformer and **RandomForestRegressor(criterion="squared_error", random_state=42, n_jobs=-1)**. Numeric features are median-imputed. Categorical features are imputed with the most frequent value and one-hot encoded with handle_unknown="ignore".

- Numeric features: pickup_lat, pickup_lon, delivery_lat, delivery_lon, distance, weight.
- Categorical features: pickup, delivery, equipment.
- Excluded predictive columns: load_id, date, market_index, quote_signal.

This baseline is the selected project benchmark because it completed the controlled chronological holdout evaluation and was used consistently for final output generation.

5. Development Benchmark and Error Analysis

Metric	Value	Scope
MAE	159.72	Recorded October chronological development holdout benchmark

The MAE value above is a development benchmark from the existing baseline evaluation. It is not Spotter's official validation score, and it was not recomputed during this final audit because that would retrain the model. score.py validates output requirements and creates the December chart; it does not calculate official prediction accuracy.

The recorded development-holdout error analysis found systematic underprediction in the high-rate segment. Long-haul observations also showed higher MAE and a more negative mean residual. These are development-holdout findings only. baseline.py contains diagnostics for equipment, weight quality/bands, distance quartiles, weekday, posted-rate bands, pickup, delivery, lane, and large absolute errors.

6. December Fixed-Input Predictions

The completed December file contains 31 rows from 2025-12-01 through 2025-12-31. All 31 existing predicted rates are identical at **823.7292**.

The selected model does not use date, and the model-used December inputs are identical across all 31 rows: Lexington pickup, Fort Wayne delivery, distance 360, Dry Van equipment, weight 32,000, and the same derived pickup/delivery coordinates. predict.py derives those coordinates from train-test.csv by combining pickup and delivery city coordinate records and checking for conflicts.

Because the model-used inputs are identical and date is excluded, the flat December prediction is expected for this baseline. This is a baseline limitation and should not be interpreted as capturing seasonality.

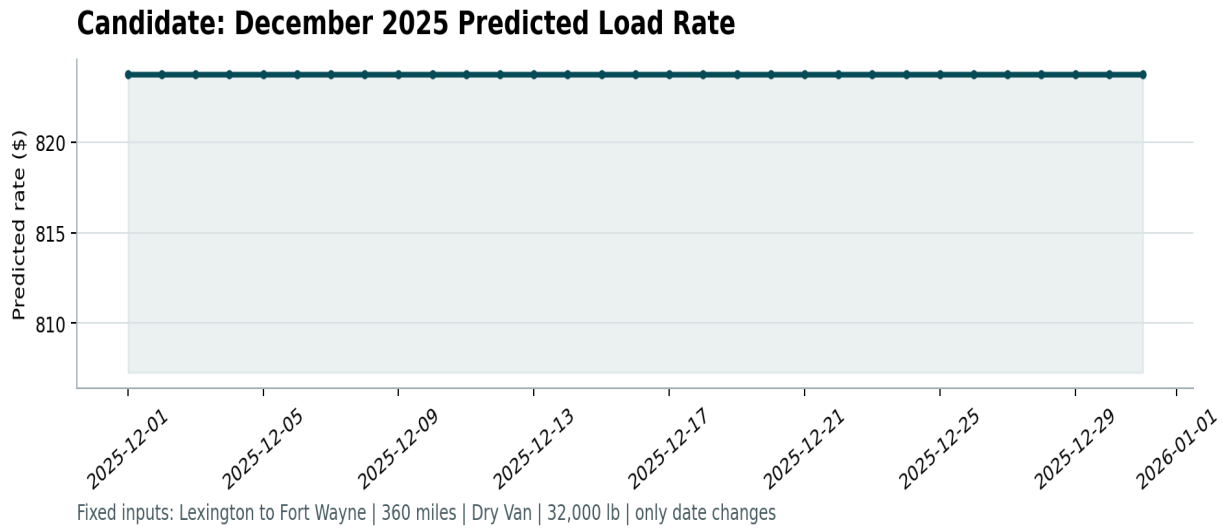


Figure 1. scorer_results/candidate_december.png generated by the provided score.py.

7. Output Validation and Reproducibility

The existing final prediction file has 12,000 rows, columns exactly load_id,predicted_rate, 12,000 unique load IDs, no missing predicted_rate values, and positive predicted rates. Its load_id order matches validation-predictions-template.csv.

For the current project layout, the successful scorer command is:

```
python score.py --predictions validation_predictions.csv --december-predictions december-chart-inputs.csv
```

The scorer validates validation_predictions.csv and december-chart-inputs.csv, creates scorer_results/candidate_december.png, and states that final validation metrics are calculated by Spotter after submission. The README example uses data/ paths, but the files in this project folder are present at the project root.

Useful reproduction commands for this project layout:

```
python baseline.py
python predict.py
python score.py --predictions validation_predictions.csv --december-predictions december-chart-inputs.csv
```

Running predict.py would regenerate prediction outputs, so it should only be used when intentionally reproducing the full pipeline.

8. Required Submission Items

- Accessible GitHub repository containing the solution code, dependencies, and run instructions.
- validation_predictions.csv containing exactly load_id,predicted_rate.
- PDF or DOCX report containing the validation approach and the fixed December chart produced by score.py.
- A 2-3 minute Loom covering key findings, data-quality issues and handling, chosen-model reasoning, training/validation split, and the most important code.

9. Scope Notes

No official Spotter validation score is known locally. This report documents the selected baseline, the recorded development benchmark, verified output files, and the scorer-produced December chart without adding unsupported metrics or new experiments.